AEA FDE Challenge: AI Call Center Agent

Overview

Build and deploy an AI agent that resolves inbound customer calls for a capacity-constrained call center, and prove it works. This is a Forward Deployed Engineer (FDE) simulation: you are dropped into a portfolio company, handed messy data, and expected to go from discovery to a monitored production prototype on your own.

The hiring partner is a private equity fund that invests in white-collar services businesses. Their FDEs work directly with exec teams at portfolio companies, scope the opportunity, build the thing, ship it, and show it is performing. This project mirrors one of the live initiatives in their portfolio: expand call center coverage without adding headcount by deflecting a majority of calls to an AI agent.

Two competencies matter most:

1. **Knowledge-base resolution** — answer the common questions callers have, grounded in the company's documentation (RAG).
2. **Scheduling** — check appointment availability against a database and complete or decline the booking.

Everything the agent cannot handle must escalate cleanly to a human.

The scenario

You are the FDE assigned to a services business whose call center is capacity constrained: staff cannot keep up with call volume, wait times are high, and the exec team wants an AI agent that handles the bulk of routine calls. The company is synthetic. The industry does not matter; the exercise does.

You will be given three things, and none of them are guaranteed to be clean:

Asset	What it is	What to expect
Call transcripts	A corpus of historical calls between customers and agents	Unlabeled. You must discover the call drivers yourself.

Knowledge base	The company's internal documentation (FAQs, policies, procedures)	Mixed formats. May include tables, images, duplicated or outdated content.
Scheduling database	A mock DB of appointment slots and bookings	Simple schema. Supports an availability check and a booking write.

The exec team has not told you which questions matter most. The transcripts will. Treat the data the way you would at a real portfolio company: start by understanding what callers actually need before building anything.

Required goals

These are the outcomes the project must demonstrate. How you get there is up to you.

1. Discovery: identify the top call drivers

- Analyze the call transcripts (clustering, classification, or another method you can defend) and identify the top 4 to 5 reasons customers call.
- Rank them by volume. Decide which ones the agent will handle and which ones it will escalate.
- Document your findings: what the drivers are, how you found them, and why you chose the scope you chose.

2. Knowledge-base resolution (RAG)

- Build a retrieval pipeline over the knowledge base that lets the agent answer the top 3 call drivers accurately.
- Make deliberate choices about chunking, embedding, and retrieval. The knowledge base may contain tables, images, or inconsistent formatting; your pipeline must account for what is actually in the docs, not what you assumed was there.
- Evaluate retrieval on its own. Show that the right documents come back before you trust the agent's answers.

3. Scheduling flow

- Connect the agent to the mock scheduling database.
- Handle the full flow: the caller asks for an appointment, the agent checks availability, and either confirms the booking or offers alternatives.
- Keep it simple. A yes/no availability check and a confirmed booking is the bar.

4. Escalation

- Anything outside the handled call drivers routes to a human.
- Escalation must be graceful: the agent recognizes it is out of scope, tells the caller what is happening, and hands off with enough context that a human can pick up without starting over.
- Do not let the agent guess. A confident wrong answer is worse than a handoff.

5. Evaluation and containment

- Build a test set from the transcripts (you will need to decide what to ask for and how to construct it).
- Measure containment rate: the share of calls fully resolved by the agent without human intervention. Target is 60 to 75 percent.
- Report accuracy on the handled drivers and the escalation rate on the rest. Show the numbers, not just the claim.

6. Deployment

- Deploy the agent as a web application at a public URL.
- The interface must be interactive: a reviewer should be able to type questions, probe edge cases, and watch the agent escalate.

7. Monitoring and feedback

- Add observability so the business can see how the agent is performing after launch: volume, containment, escalations, failures, latency.
- Include a feedback or learning loop. Logged conversations, flagged failures, or human corrections should feed back into improving the system over time. Describe how the loop works even if you only implement the first step.

Extra credit: voice

The baseline is a chat interface.

If you want to stand out, make the agent answer actual phone calls.

Voice adds real complexity: speech-to-text and text-to-speech, latency budgets, turn-taking, interruptions, and confirming details like names and times that transcription gets wrong. The reviewer knows this. A working voice agent, even a rough one, is a strong signal. A polished chat agent with no voice is still a complete submission.

Deliverables

Submit three things:

1. **Deployed URL** — a live web app the reviewer can interact with directly. They will ask their own questions and try to break it, so make sure edge cases and escalation behave.
2. **Walkthrough video** — a 3 to 4 minute voiceover covering your critical design choices: how you found the call drivers, why you chose your chunking and retrieval approach, how escalation works, and what your eval numbers say. Design reasoning, not a feature tour.
3. **Source repository** — code, eval set, and a README that explains how to run it and what the containment numbers were.

What we are looking for

The reviewer is hiring an FDE, not a prompt engineer. The question is whether you can be dropped into an unfamiliar business, figure out what matters, build something that works, and prove it.

- **Judgment on scope.** Did you pick the right 3 drivers to handle and the right ones to escalate? Can you explain why?
- **Honesty about the data.** Did you look at what was actually in the knowledge base and transcripts, or did you assume? Did you catch the traps?
- **Confidence backed by evidence.** Is the containment number real? Does the test set reflect real calls? Would you stake your reputation on it in front of a CEO?
- **Production thinking.** Could this run for a month without you? Would the business know if it broke?
- **Clean failure.** When the agent does not know, does it say so and hand off well?

A smaller system that works and is measured beats a larger one that is neither.